

Shortest GCD Paths

Tutorial suy luận và lời giải tối ưu

Competitive Programming Notes

June 10, 2026

Contents

1 Bài toán	2
2 Phân tích giới hạn	2
3 Phân tích một cạnh	2
4 Loại bỏ phần chung của a và b	3
5 Từ đường đi sang bài toán phân nhóm	3
6 Chứng minh hai chiều	3
6.1 Từ đường đi bất kỳ sang phân nhóm	3
6.2 Từ phân nhóm sang đường đi hợp lệ	4
7 Vì sao không vượt quá n?	4
8 DP trên các ước	5
9 Độ phức tạp	5
10 Dry run	6
10.1 Ví dụ 1	6
10.2 Ví dụ 2	6
10.3 Ví dụ 3	6
11 Những hướng sai dễ mắc	7
12 Pattern cho bài khác	7
13 Kiểm chứng bằng brute force	7
14 Code C++17	8
14.1 Solution	8
14.2 Bruteforce checker	9
15 Tóm tắt để ôn nhanh	11

1 Bài toán

Ta có đồ thị vô hướng đầy đủ gồm n đỉnh, đánh số từ 1 đến n . Với mỗi cặp đỉnh phân biệt u, v , trọng số cạnh là

$$w(u, v) = \frac{\max(u, v)}{\gcd(u, v)}.$$

Cần tìm độ dài đường đi ngắn nhất từ a đến b , với

$$2 \leq n \leq 10^9, \quad 1 \leq a, b \leq n, \quad a \neq b.$$

Ví dụ:

Input	Output
10 9 8	7
100 16 27	10
100 55 11	5

2 Phân tích giới hạn

Vì $n \leq 10^9$, các hướng trực tiếp đều không khả thi.

- Không thể dựng toàn bộ đồ thị vì số cạnh là $\Theta(n^2)$.
- Không thể chạy Dijkstra trên $1, \dots, n$, vì chỉ riêng số đỉnh đã quá lớn.
- Không thể DP theo giá trị đỉnh, vì mảng kích thước 10^9 không thực tế.
- Không thể duyệt mọi đỉnh trung gian.

Nhận xét. Khi miền giá trị rất lớn nhưng công thức transition chứa gcd, chia hết, tỉ lệ hoặc thừa số, ta nên thử rời bỏ miền giá trị $1, \dots, n$, và chuyển sang trạng thái theo ước số hoặc thừa số nguyên tố.

Với bài này, điểm đáng nhìn nhất không phải là đồ thị đầy đủ, mà là công thức trọng số. Ta sẽ bóc nó ra.

3 Phân tích một cạnh

Lấy hai số u, v . Đặt

$$g = \gcd(u, v), \quad u = gx, \quad v = gy,$$

trong đó

$$\gcd(x, y) = 1.$$

Khi đó

$$w(u, v) = \frac{\max(gx, gy)}{g} = \max(x, y).$$

Nghĩa là một bước đi từ u sang v có thể hiểu như sau:

- bỏ hệ số x ;
- thêm hệ số y ;
- chi phí là $\max(x, y)$.

Ví dụ:

$$16 \rightarrow 24.$$

Ta có

$$\gcd(16, 24) = 8, \quad 16 = 8 \cdot 2, \quad 24 = 8 \cdot 3.$$

Vậy chi phí cạnh là

$$\max(2, 3) = 3.$$

4 Loại bỏ phần chung của a và b

Đặt

$$g = \gcd(a, b), \quad A = \frac{a}{g}, \quad B = \frac{b}{g}.$$

Khi đó

$$\gcd(A, B) = 1.$$

Phần g là phần chung đã có ở cả điểm đầu và điểm cuối. Trong một lời giải tối ưu, ta không cần xem nó là thứ phải thay đổi. Phần thực sự cần xử lý là:

- loại bỏ toàn bộ thừa số thuộc A ;
- thêm toàn bộ thừa số thuộc B .

Vì A và B nguyên tố cùng nhau, mọi thừa số thuộc A và mọi thừa số thuộc B cũng nguyên tố cùng nhau. Điều này làm cho chi phí của từng nhóm rất sạch.

5 Từ đường đi sang bài toán phân nhóm

Giả sử một bước loại bỏ hệ số x và thêm hệ số y . Chi phí là

$$\max(x, y).$$

Nếu cả đường đi gồm các bước

$$(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m),$$

thì phần bị loại bỏ phải nhân lại thành A :

$$x_1 x_2 \cdots x_m = A,$$

và phần được thêm vào phải nhân lại thành B :

$$y_1 y_2 \cdots y_m = B.$$

Tổng chi phí là

$$\sum_{i=1}^m \max(x_i, y_i).$$

Nhận xét. Bài toán đường đi ngắn nhất trên đồ thị không lồ đã biến thành bài toán phân nhóm thừa số: chia thừa số của A và B thành các nhóm, rồi ghép từng nhóm của A với một nhóm của B .

6 Chứng minh hai chiều

Đây là phần quan trọng nhất. Nếu bỏ qua chứng minh này, ta rất dễ viết một DP đẹp nhưng không chắc nó đang giải đúng bài gốc.

6.1 Từ đường đi bất kỳ sang phân nhóm

Xét một đường đi bất kỳ

$$a = z_0, z_1, \dots, z_m = b.$$

Với bước i , đặt

$$h_i = \gcd(z_{i-1}, z_i), \quad r_i = \frac{z_{i-1}}{h_i}, \quad s_i = \frac{z_i}{h_i}.$$

Khi đó $\gcd(r_i, s_i) = 1$, và chi phí bước i là

$$\max(r_i, s_i).$$

Nhìn theo số mũ nguyên tố. Với một prime p thuộc A , tổng số lần số mũ của p giảm trừ tổng số lần nó tăng đúng bằng số mũ của p trong A . Nếu một prime thuộc A bị giảm rồi lại tăng, phần tăng đó là dao động trung gian, không đóng góp vào biến đổi ròng từ a sang b .

Vì vậy, từ các lần giảm của prime thuộc A , ta chọn đúng lượng giảm ròng cần thiết để tạo các x_i . Tương tự, từ các lần tăng của prime thuộc B , ta chọn đúng lượng tăng ròng cần thiết để tạo các y_i .

Khi đó:

$$x_i \mid r_i, \quad y_i \mid s_i.$$

Do đó

$$\max(x_i, y_i) \leq \max(r_i, s_i).$$

Đồng thời các phần được chọn nhân lại đúng bằng

$$\prod_i x_i = A, \quad \prod_i y_i = B.$$

Suy ra từ mọi đường đi ta lấy được một cách phân nhóm có tổng chi phí không lớn hơn đường đi ban đầu.

6.2 Từ phân nhóm sang đường đi hợp lệ

Ngược lại, giả sử ta có các cặp

$$(x_1, y_1), \dots, (x_m, y_m)$$

thỏa

$$\prod_i x_i = A, \quad \prod_i y_i = B.$$

Vì A và B nguyên tố cùng nhau, nên $\gcd(x_i, y_i) = 1$ với mọi i .

Từ một trạng thái hiện tại z , nếu $x_i \mid z$, ta đi sang

$$z' = \frac{z}{x_i} \cdot y_i.$$

Khi đó

$$\gcd(z, z') = \frac{z}{x_i},$$

và cạnh có chi phí đúng bằng

$$\max(x_i, y_i).$$

Do các x_i nhân lại thành A , ta có thể loại bỏ chúng lần lượt từ $a = gA$. Do các y_i nhân lại thành B , sau cùng ta đến $gB = b$.

7 Vì sao không vượt quá n ?

Còn một ràng buộc phải xử lý: mọi đỉnh trung gian phải nằm trong $1, \dots, n$.

Phân loại các bước:

- bước giảm hoặc giữ nguyên: $y_i \leq x_i$;
- bước tăng: $y_i > x_i$.

Ta có thể sắp xếp tất cả bước giảm trước, bước tăng sau. Lý do là các tỉ lệ

$$\frac{y_i}{x_i}$$

nhân với nhau, nên thứ tự không ảnh hưởng đến tích cuối. Tổng chi phí cũng không phụ thuộc vào thứ tự.

Sau khi sắp:

- giai đoạn giảm không bao giờ vượt quá a ;
- giai đoạn tăng là một dãy tăng dần và kết thúc tại b , nên không vượt quá b .

Vậy mọi đỉnh trung gian không vượt quá

$$\max(a, b) \leq n.$$

Điều kiện chia hết cũng được giữ: tại thời điểm xử lý x_i , các thừa số của x_i vẫn còn nằm trong phần A chưa bị loại bỏ. Những thừa số đã thêm từ B không gây cản trở vì A và B nguyên tố cùng nhau.

8 DP trên các ước

Ta liệt kê tất cả ước của A và B . Định nghĩa

$$dp[x][y]$$

là chi phí nhỏ nhất để xử lý xong phần x của A và phần y của B , trong đó

$$x \mid A, \quad y \mid B.$$

Base:

$$dp[1][1] = 0.$$

Giả sử bước cuối xử lý nhóm

$$d \mid x, \quad e \mid y, \quad (d, e) \neq (1, 1).$$

Trước bước cuối, ta đã xử lý

$$\frac{x}{d}, \quad \frac{y}{e}.$$

Do đó

$$dp[x][y] = \min_{\substack{d \mid x, e \mid y \\ (d, e) \neq (1, 1)}} \left(dp \left[\frac{x}{d} \right] \left[\frac{y}{e} \right] + \max(d, e) \right).$$

Ta duyệt các ước theo thứ tự tăng dần. Nếu $(d, e) \neq (1, 1)$, thì hoặc $x/d < x$, hoặc $y/e < y$. Vì vậy trạng thái phụ đã được tính trước.

9 Độ phức tạp

Gọi

$$D_A = \tau(A), \quad D_B = \tau(B),$$

trong đó $\tau(x)$ là số ước của x .

Với $x \leq 10^9$, số ước lớn nhất là

$$1344,$$

đạt tại

$$735134400 = 2^6 \cdot 3^3 \cdot 5^2 \cdot 7 \cdot 11 \cdot 13 \cdot 17.$$

Số trạng thái DP là

$$D_A D_B.$$

Ta tiến xử lý, với mỗi ước x , danh sách các cặp

$$(d, x/d), \quad d \mid x.$$

Nếu

$$S(A) = \sum_{x \mid A} \tau(x),$$

thì tổng số transition thực sự là

$$S(A)S(B).$$

Với phân tích thừa số

$$A = \prod p_i^{\alpha_i},$$

ta có

$$S(A) = \prod_i \frac{(\alpha_i + 1)(\alpha_i + 2)}{2}.$$

Riêng với một số không vượt quá 10^9 , $S(x)$ lớn nhất là 136080. Trong bài này A và B nguyên tố cùng nhau sau khi chia $\gcd(a, b)$, nên hai phía không thể đồng thời đạt cấu hình xấu nhất dùng cùng các prime nhỏ.

Trong thực tế, số transition đủ nhỏ cho một test. Việc sinh ước chỉ cần $O(\sqrt{A} + \sqrt{B})$, với $\sqrt{10^9} < 31623$.

10 Dry run

10.1 Ví dụ 1

$$n = 10, \quad a = 9, \quad b = 8.$$

Ta có $\gcd(9, 8) = 1$, nên

$$A = 9, \quad B = 8.$$

Một phân nhóm tối ưu là

$$(3, 2), (3, 4).$$

Tổng chi phí:

$$\max(3, 2) + \max(3, 4) = 3 + 4 = 7.$$

Đường đi tương ứng:

$$9 \rightarrow 6 \rightarrow 8.$$

Kiểm tra:

$$w(9, 6) = \frac{9}{3} = 3, \quad w(6, 8) = \frac{8}{2} = 4.$$

10.2 Ví dụ 2

$$n = 100, \quad a = 16, \quad b = 27.$$

Ta có

$$A = 16, \quad B = 27.$$

Một phân nhóm tối ưu:

$$(2, 3), (2, 3), (4, 3).$$

Tổng chi phí:

$$3 + 3 + 4 = 10.$$

Đường đi:

$$16 \rightarrow 24 \rightarrow 36 \rightarrow 27.$$

Kiểm tra:

$$w(16, 24) = 3, \quad w(24, 36) = 3, \quad w(36, 27) = 4.$$

10.3 Ví dụ 3

$$n = 100, \quad a = 55, \quad b = 11.$$

Ta có

$$g = 11, \quad A = 5, \quad B = 1.$$

Chỉ cần bỏ hệ số 5, nên đáp án là 5. Đường đi trực tiếp:

$$55 \rightarrow 11, \quad w(55, 11) = 5.$$

11 Những hướng sai dễ mắc

- **Dijkstra trực tiếp trên $1, \dots, n$.** Không chạy được vì $n \leq 10^9$.
- **Chỉ xét đường đi trực tiếp.** Sai với $16 \rightarrow 27$: trực tiếp tốn 27, nhưng $16 \rightarrow 24 \rightarrow 36 \rightarrow 27$ tốn 10.
- **Chỉ xét đỉnh là ước của ab mà không chứng minh.** Ý tưởng gần đúng, nhưng cần chứng minh bằng phân tích thừa số rỗng.
- **Dijkstra trên cặp ước với transition tùy ý.** Làm được nhưng thừa; DP đã có thứ tự topo tự nhiên.
- **Greedy ghép thừa số nhỏ nhất.** Không có tính chất local-choice rõ ràng; grouping cần tối ưu tổng $\max(d, e)$.
- **Greedy ghép thừa số gần nhau nhất.** Có thể bị kẹt vì một nhóm lớn đôi khi nên ghép với nhiều nhóm nhỏ.
- **Bỏ qua giới hạn đỉnh trung gian.** Cần chứng minh sắp bước giảm trước, tăng sau để mọi đỉnh không vượt quá $\max(a, b)$.

12 Pattern cho bài khác

Checklist nhận diện:

1. Constraint cực lớn nên không duyệt miền giá trị.
2. Phân tích công thức của một transition.
3. Tách phần gcd chung nếu có.
4. Diễn giải transition thành thao tác trên thừa số.
5. Xác định tài nguyên còn lại để xây dựng state.
6. DP trên ước thay vì DP trên giá trị.
7. Dùng tính giao hoán để đổi thứ tự thao tác và xử lý ràng buộc trung gian.

13 Kiểm chứng bằng brute force

Ta viết checker riêng:

- Với $n \leq 30$, dựng đầy đủ graph.
- Chạy Dijkstra thật trên n đỉnh.
- So sánh với lời giải DP cho mọi cặp $a \neq b$.
- Sau đó random thêm với $n \leq 100$.

Kết quả khi chạy checker trong thư mục dự án:

All tests passed, Total tests: 13990.

14 Code C++17

14.1 Solution

```
#include <bits/stdc++.h>
#define ll long long
#define fi first
#define se second
#define pb emplace_back
#define ii pair<int, int>
#define MASK(i) (1LL << (i))
#define BIT(x, i) (((x) >> (i)) & 1)
#define sz(s) (int)((s).size())
#define all(a) a.begin(), a.end()
#define FOR(i, a, b) for (int i = (a), _b = (b); i <= _b; ++i)
#define F0R(i, b) for (int i = 0, _b = (b); i < _b; ++i)
#define F0Rd(i, a, b) for (int i = (a), _b = (b); i >= _b; --i)
#define F0Rd(i, b) for (int i = (b)-1; i >= 0; i--)
using namespace std;

template<typename T1, typename T2> bool ckmax(T1 &A, const T2 &B){if(A < B){A = B;return true;}
return false;}
template<typename T1, typename T2> bool ckmin(T1 &A, const T2 &B){if(A > B){A = B;return true;}
return false;}

const int MOD = (int)1e9 + 7;
const int N = 3e5 + 10, M = 18;
const int INF = (int)2e9 + 11;
const long long LINF = (long long)4e18 + 11;

vector<ll> get_divs(ll x){
    vector<ll> divs;
    for(ll d = 1; d * d <= x; ++d) if(x % d == 0){
        divs.pb(d);
        if(d * d != x) divs.pb(x / d);
    }
    sort(all(divs));
    return divs;
}

vector<vector<ii>> build_splits(const vector<ll> &divs){
    int n = sz(divs);
    unordered_map<ll, int> id;
    id.reserve(n * 2 + 1);
    F0R(i, n) id[divs[i]] = i;
    vector<vector<ii>> splits(n);
    F0R(i, n){
        ll x = divs[i];
        F0R(j, n) if(x % divs[j] == 0){
            ll rest = x / divs[j];
            auto it = id.find(rest);
            if(it != id.end()) splits[i].pb(j, it->se);
        }
    }
    return splits;
}

ll solve_one(ll a, ll b){
    ll g = gcd(a, b);
    ll A = a / g, B = b / g;
    vector<ll> da = get_divs(A), db = get_divs(B);
    auto sa = build_splits(da);
    auto sb = build_splits(db);
    int na = sz(da), nb = sz(db);
    vector<ll> dp(na * nb, LINF);
```

```

    auto at = [&](int i, int j)->ll&{ return dp[i * nb + j]; };
    at(0, 0) = 0;
    FOR(i, na) FOR(j, nb){
        if(i == 0 && j == 0) continue;
        ll best = LINF;
        for(auto pa : sa[i]) for(auto pb : sb[j]){
            int di = pa.fi, pi = pa.se;
            int ei = pb.fi, pj = pb.se;
            if(di == 0 && ei == 0) continue;
            ckmin(best, at(pi, pj) + max(da[di], db[ei]));
        }
        at(i, j) = best;
    }
    return at(na - 1, nb - 1);
}

void sol(void){
    ll n, a, b;
    cin >> n >> a >> b;
    cout << solve_one(a, b) << '\n';
}

signed main(void){
    #define TASK "nhap"
    if(fopen(TASK".inp", "r")){
        freopen(TASK".inp", "r", stdin);
        freopen(TASK".out", "w", stdout);
    }
    ios_base::sync_with_stdio(false); cin.tie(NULL); cout.tie(NULL);
    int t = 1;
    while(t--) sol();
    cerr << "\nTime elapsed: " << 1000 * clock() / CLOCKS_PER_SEC << " ms\n";
}

```

14.2 Bruteforce checker

```

#include <bits/stdc++.h>
#define ll long long
#define fi first
#define se second
#define pb emplace_back
#define ii pair<int, int>
#define MASK(i) (1LL << (i))
#define BIT(x, i) (((x) >> (i)) & 1)
#define sz(s) (int)((s).size())
#define all(a) a.begin(), a.end()
#define FOR(i, a, b) for (int i = (a), _b = (b); i <= _b; ++i)
#define F0R(i, b) for (int i = 0, _b = (b); i < _b; ++i)
#define FORD(i, a, b) for (int i = (a), _b = (b); i >= _b; --i)
#define F0RD(i, b) for (int i = (b)-1; i >= 0; i--)
using namespace std;

template<typename T1, typename T2> bool ckmax(T1 &A, const T2 &B){if(A < B){A = B;return true;}
return false;}
template<typename T1, typename T2> bool ckmin(T1 &A, const T2 &B){if(A > B){A = B;return true;}
return false;}

const int MOD = (int)1e9 + 7;
const int N = 3e5 + 10, M = 18;
const int INF = (int)2e9 + 11;
const long long LINF = (long long)4e18 + 11;

vector<ll> get_divs(ll x){
    vector<ll> divs;

```

```

    for(ll d = 1; d * d <= x; ++d) if(x % d == 0){
        divs.pb(d);
        if(d * d != x) divs.pb(x / d);
    }
    sort(all(divs));
    return divs;
}

vector<vector<ii>> build_splits(const vector<ll> &divs){
    int n = sz(divs);
    unordered_map<ll, int> id;
    id.reserve(n * 2 + 1);
    FOR(i, n) id[divs[i]] = i;
    vector<vector<ii>> splits(n);
    FOR(i, n){
        ll x = divs[i];
        FOR(j, n) if(x % divs[j] == 0){
            ll rest = x / divs[j];
            auto it = id.find(rest);
            if(it != id.end()) splits[i].pb(j, it->se);
        }
    }
    return splits;
}

ll dp_answer(ll a, ll b){
    ll g = gcd(a, b);
    ll A = a / g, B = b / g;
    vector<ll> da = get_divs(A), db = get_divs(B);
    auto sa = build_splits(da);
    auto sb = build_splits(db);
    int na = sz(da), nb = sz(db);
    vector<ll> dp(na * nb, LINF);
    auto at = [&](int i, int j)->ll&{ return dp[i * nb + j]; };
    at(0, 0) = 0;
    FOR(i, na) FOR(j, nb){
        if(i == 0 && j == 0) continue;
        ll best = LINF;
        for(auto pa : sa[i]) for(auto pb : sb[j]){
            int di = pa.fi, pi = pa.se;
            int ei = pb.fi, pj = pb.se;
            if(di == 0 && ei == 0) continue;
            ckmin(best, at(pi, pj) + max(da[di], db[ei]));
        }
        at(i, j) = best;
    }
    return at(na - 1, nb - 1);
}

ll brute_answer(int n, int a, int b){
    vector<ll> dist(n + 1, LINF);
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq;
    dist[a] = 0;
    pq.push({0, a});
    while(!pq.empty()){
        auto [du, u] = pq.top(); pq.pop();
        if(du != dist[u]) continue;
        if(u == b) return du;
        FOR(v, 1, n) if(v != u){
            ll w = max(u, v) / gcd(u, v);
            if(ckmin(dist[v], du + w)) pq.push({dist[v], v});
        }
    }
    return dist[b];
}

```

```

void require_equal(int n, int a, int b, long long &tests){
    ll fast = dp_answer(a, b);
    ll slow = brute_answer(n, a, b);
    ++tests;
    if(fast != slow){
        cout << "Counterexample found\n";
        cout << "n = " << n << ", a = " << a << ", b = " << b << '\n';
        cout << "dp = " << fast << ", brute = " << slow << '\n';
        exit(0);
    }
}

void sol(void){
    long long tests = 0;
    FOR(n, 2, 30) FOR(a, 1, n) FOR(b, 1, n) if(a != b) require_equal(n, a, b, tests);
    mt19937 rng(712367);
    FOR(it, 5000){
        int n = 2 + (rng() % 99);
        int a = 1 + (rng() % n);
        int b = 1 + (rng() % n);
        if(a == b) b = b % n + 1;
        require_equal(n, a, b, tests);
    }
    cout << "All tests passed\n";
    cout << "Total tests: " << tests << '\n';
}

signed main(void){
    ios_base::sync_with_stdio(false); cin.tie(NULL); cout.tie(NULL);
    int t = 1;
    while(t--) sol();
    cerr << "\nTime elapsed: " << 1000 * clock() / CLOCKS_PER_SEC << " ms\n";
}

```

15 Tóm tắt để ôn nhanh

- Cảnh $u \rightarrow v$: viết $u = gx, v = gy, \gcd(x, y) = 1$, chi phí là $\max(x, y)$.
- Chia $g = \gcd(a, b)$, còn lại $A = a/g, B = b/g$, với $\gcd(A, B) = 1$.
- Cần bỏ các thừa số của A , thêm các thừa số của B .
- Một lời giải là phân nhóm (x_i, y_i) sao cho $\prod x_i = A, \prod y_i = B$, chi phí $\sum \max(x_i, y_i)$.
- DP trên cặp ước:

$$dp[x][y] = \min_{d|x, e|y, (d,e) \neq (1,1)} dp[x/d][y/e] + \max(d, e).$$

- Ràng buộc $1..n$ không bị phá vì có thể sắp bước giảm trước, tăng sau.
- Luôn brute force các bài kiểu này nếu có thể: graph nhỏ dựng được, lời giải tối ưu khó chứng minh một phát ănn ngay.